Proposal: Cascabel — task-difficulty model routing for Pi

Status: working prototype, live-verified · **Author:** Reaan06 · **Date:** 2026-11-12 **Repo:** `github.com/Reaan06/cascabel` · **License:** MIT

The problem

Coding agents spend most of their budget on the default answer to two questions: *which model runs this task?* and *at what thinking effort?* The usual answer is "the same model at the same effort for everything" — top-tier reasoning for renaming a variable, and the same flat effort for a migration that deserves more.

Production teams that route by task difficulty report **35–65% lower cost at the same quality** compared to always running the top model (Model Routing Playbook, AI LEVELS, 2026). The catch: generic agent harnesses have no per-task routing hook. You either re-route manually all day, or fork the harness.

The proposal

Cascabel is a Pi extension that gives ONE worker a difficulty-based model tier and a capped escalation cascade — with zero new agent definitions and zero classification calls.

The loop, once installed:



```
                                        |

   quick   gemini-3.8-flash-high @ medium            FAIL: one retry in
tier,

   build   deepseek-v4-flash     @ high              then exactly one tier
step

   deep    glm5.3                @ max               up, then a human FAIL
```

Design rules that make it trustworthy rather than clever:

1. **The child is your worker.** Instructions are live-read at every spawn from your existing worker agent file — Cascabel creates no agent definitions and clones nothing.
2. **Difficulty comes from exploration, never from a guess.** No classifier LLM call inside the router; the caller (your explorer/orchestrator) states `quick | build | deep`, optionally with a named risk. A named risk never routes below `build`.
3. **Fail-closed everywhere.** Unknown difficulty, missing tier, missing config, missing worker prompt, missing verification verdict — all hard errors with actionable messages. Cascabel never invents a model.
4. **The cap is the feature.** One retry per tier, one step up (quick→build→deep). A cascade without a cap burns 4× per failed cheap run — the exact failure mode the routing literature warns about.
5. **Verification is the child's contract.** Every routed run must end with a machine-parseable `cascabel_verdict: pass|fail` line backed by executed checks (tests, types, lint). A missing verdict counts as fail.
6. **Every attempt leaves a receipt.** One JSON line per attempt (tier, model, effort, duration, verdict reason) in `~/.pi/cascabel/logs/attempts.jsonl` — the raw material for the playbook rule "log every miss and turn it into a rule" (cost per solved task, not per token).

Evidence (measured on the working prototype)

Check	Result
Unit suite (tier resolution, capped cascade, config fail-closed, RPC framing, receipts)	**69 passing**
Real end-to-end smoke (headless child spawn, RCP handshake, verdict parse)	**PASS in ~9–12 s**
Live loop with a real bug proposal, fresh session, no hand-holding	**1 m 07 s total**: session classified the bug, routed, child fixed the repo, `npm test` green, verdict pass
Receipt recorded for that run	`{tier: quick, model: gemini-3.8-flash-high, ok: true, 19.5 s}`
Model availability discipline	Tiers verified resolvable headless via `get_available_models` before shipping; unknown models fail fast with receipts

What it deliberately does NOT do

- No automatic model auto-router/classifier: plain rules beat magic, and a classifier adds cost and a new failure surface.
- No mid-task model switching: each routed task is a fresh child session, so prompt caches stay warm both in the parent and in the child tier.
- No replacement for the host harness: Cascabel complements the orchestrator's existing delegation and verification discipline; the parent stays owner of specs, commits and review.

Installation and use

```
pi install git:github.com/Reaan06/cascabel   # or npm:/local path
```

Then, in any new session, the tool `cascabel_route_run` appears. Configure the ladder in `~/.pi/cascabel/routings.json` (defaults work out of the box; the file is authoritative and merged over defaults, invalid entries fail closed).

Roadmap

- **v0.2 (this proposal):** package polish, CI, docs, public release.
- **v0.3:** escalation telemetry in the tool result tokens/cost fields; routing suggestions generated from the receipts log.
- **v0.4:** optional classifier lane for unattended batch routing (opt-in, measured against a baseline before it earns trust).

Risks and mitigations

- **False passes on the cheap tier** eat the savings: mitigated by the verdict contract (fail-closed), the retry cap, and receipts that make false-pass rate measurable per tier.
- **Runner coupling:** Cascabel depends only on Pi's documented CLI flags, RPC protocol and extension API — never on harness internals; a missing optional extension degrades to a clear error instead of a silent guess.
- **Headless auto-approval:** routed children execute tools without interactive approval (they cannot answer prompts); the boundary is the delegated task text and the caller's existing permissions.

Recommendation

Adopt Cascabel as the routing layer for delegated implementation work where a reliable check exists (tests, types, lint, schema). Start conservative: route the tasks you would have re-routed manually anyway, watch `attempts.jsonl` for a week, and tune the ladder from measured cost per solved task — not per token.

Source of the pattern: Model Routing Playbook (AI LEVELS, 2026). All numbers in this document come from this repository's own test suite and live QA runs, reproducible via `npm test` and `npm run smoke`.